SMSystem - Sales and Inventory Management System

Project Documentation & Technical Overview

---

1. Project Overview

SMSystem is a modern, full-stack Point of Sale (POS) and Inventory Management solution designed to streamline wholesale and retail operations. It provides a robust, real-time suite of tools for processing sales, managing inventory levels, handling supplier procurements, and tracking business expenses. The system is built with a focus on speed, accuracy, and rigorous role-based security.

---

2. Technology Stack

The application is built using a modern decoupled architecture, ensuring scalability, maintainability, and high performance.

Frontend (Client-Side)

- **Framework**: React.js with TypeScript for type-safe, dynamic user interfaces.
- **Styling**: Tailwind CSS for responsive, clean, and professional aesthetics.
- **Routing**: React Router DOM for seamless Single Page Application (SPA) navigation.
- **Icons**: Lucide React for consistent and crisp operational iconography.
- **Build Tool**: Vite for lightning-fast development and optimized production builds.

Backend (Server-Side)

- **Language**: Go (Golang) for high-concurrency and blazing-fast API responses.
- **Web Framework**: Gin HTTP web framework, known for its performance and router efficiency.
- **ORM**: GORM for robust, secure, and intuitive database interactions.
- **Authentication**: JWT (JSON Web Tokens) combined with `bcrypt` password hashing.
- **Database**: MySQL for reliable, ACID-compliant relational data storage.

---

3. Core Modules & Features

3.1 Point of Sale (POS) & Checkout

- **Instant Checkout**: Real-time product search, barcode-ready interface, and cart management.
- **Flexible Payments**: Handles diverse payment protocols including Cash, Credit Card, GCash, Bank Transfer, and Check.
- **Dynamic Taxation**: Built-in support for Withholding Tax and VAT calculations.
- **Advanced Receipt Printing**: Flexible receipt generation supporting both **Sales Invoices (S.I.)** and **Delivery Receipts (D.R.)**, utilizing absolute-positioned CSS for pixel-perfect printing onto pre-printed physical forms.

3.2 Inventory Management

- **Catalog Matrix**: Full CRUD (Create, Read, Update, Delete) operations for Products, logically grouped by Categories and Brands.
- **Stock Oversight**: Real-time stock level tracking with automatic low-stock alerts explicitly visible on the dashboard.

- **Bulk Imports**: System functionalities designed to handle bulk importing of product catalogs for rapid onboarding.

3.3 Procurement & Suppliers

- **Supplier Repository**: Manage localized and international vendor contact details and business information.
- **Purchase Orders (PO)**: Create localized purchase orders to restock inventory. The system formally tracks PO status and automatically updates product stock quantities upon registering orders as "Received".

3.4 Financials & Expense Tracking

- **Overhead Management**: Track operational costs categorized cleanly by Rent, Salary, Utilities, Supplies, and Inventory.
- **Currency Compliance**: Fully localized currency formatting (₱ - Philippine Peso).
- **Direct Inventory Linking**: Seamless linkage allows logging restock expenses that directly map back to specific product inventory.

3.5 Customer Management

- **Client Database**: Record comprehensive customer details including TIN, Business Style, physical addresses, and contact numbers.
- **Fast Mapping**: Instantly search and attach returning customers during the POS checkout process to speed up queue times.

3.6 Analytics & Executive Dashboard

- **KPI Tracking**: Real-time visual statistics displaying Total Sales revenue, Total Orders processed, Active Products count, and Low Stock Warnings.
- **Performance Graphs**: Visual representation of recent sales performance mapping revenue against time.

3.7 System Auditing

- **Activity Logs**: Comprehensive, non-deletable activity logging for highly sensitive operations (e.g., role modification, user deletion, high-value payments, inventory overrides).
- **Traceability**: Every sensitive action is tracked by the executing User, Action Type, Target Entity, and the physical IP Address.

---

4. User Roles & Access Control

The system employs a strict, airtight Role-Based Access Control (RBAC) mechanism to ensure business security:

- **Administrator (Owner)**: Full system control. Can access financial overviews, manage staff roles, reset employee passwords, view raw audit logs, and modify core configurations.
- **Cashier (Staff)**: Frontline access prioritized for POS checkout, Customer management, and viewing Products. Restricted from viewing global dashboard financials or altering core system settings.
- **Purchasing (Inventory)**: Focused access granted specifically for managing Suppliers, generating Purchase Orders, and overseeing stock levels.
- **Unverified User**: The default sandbox role. New signups receive read-only or blocked access until an Administrator grants them an official operational role.

---

5. Security & Architecture

- **Stateless Authentication**: JWT-based authentication ensures APIs are protected without maintaining heavy server sessions.
- **Data Protection**: All staff and user passwords are irreversibly hashed using the `bcrypt` cryptographic algorithm.
- **Middleware Protection**: Specialized Go middlewares proactively intercept incoming requests and validate user roles (e.g., `RequireRole("admin")`) before any database operations are triggered.
- **API-Driven Decoupling**: Frontend and Backend talk strictly via RESTful JSON endpoints, meaning the backend can be attached to mobile apps or external integrations in the future with zero rewrites.
- **Relational Integrity**: The MySQL database employs strict Foreign Key constraints ensuring that deleting a category, brand, or user operates gracefully without leaving orphaned data.

---

6. Getting Started & Setup Guide

Follow these instructions to run the SMSystem locally for development and testing.

Prerequisites

- Node.js (v18 or higher) for React frontend.
- Go (Golang) (v1.20 or higher) for the backend API.
- MySQL database instance.

6.1 Database Setup

1. Open your MySQL terminal or GUI (like MySQL Workbench).
2. Create the system database:

   ```
   CREATE DATABASE IF NOT EXISTS smsystem_db;
   ```

3. The Go backend uses GORM's AutoMigrate, so all tables, keys, and schemas will be created automatically the first time the server starts!

6.2 Backend (Go API)

1. Navigate to the backend directory:

   ```
   cd backend
   ```

2. Configure your environment variables: Ensure `config.json` or `.env` has your database credentials (e.g., `root:password@tcp(127.0.0.1:3306)/smsystem_db?parseTime=true`).
3. Download Go modules:

   ```
   go mod download
   ```

4. Start the backend server:

   ```
   go run cmd/server/main.go
   ```

   The server will typically start on `http://localhost:8080` .

6.3 Frontend (React)

1. Open a new terminal and navigate to the frontend directory:

```
cd frontend
```

2. Install application dependencies:

```
npm install
```

3. Start the Vite development server:

```
npm run dev
```

4. Access the application in your browser at `http://localhost:5173`. Make sure the frontend `.env` points its `VITE_API_URL` to the Go backend (`http://localhost:8080`).

---

7. Building for Production (Windows .exe & Web)

If you wish to deploy the system or run it without starting development servers, you can compile the codebase into optimized, production-ready artifacts.

7.1 Compiling the Go Backend to a Windows Executable (.exe)

You can compile the backend into a standalone `.exe` file that can be natively run on any Windows machine.

1. Open your terminal or Command Prompt and navigate to the `backend` folder:

```
cd backend
```

2. Run the Go build command. This tells Go to compile a Windows executable file named `SMSystemServer.exe`:

```
env GOOS=windows GOARCH=amd64 go build -o SMSystemServer.exe
cmd/server/main.go
```

   (Note: If you are compiling directly on a Windows PC, you can simply run `go build -o SMSystemServer.exe cmd\server\main.go` without the environment variables).
3. You will now see a file named `SMSystemServer.exe`. You can double-click this file on your Windows machine to start the production API server.

7.2 Compiling the React Web Frontend

You must also compile the React application into static HTML, CSS, and JS files for optimal performance.

1. Navigate to the `frontend` folder:

```
cd frontend
```

2. Run the Vite build command:

```
npm run build
```

3. This will create a new directory named `dist/` inside your frontend folder. This directory contains your entire compiled application.
4. You can serve this `dist/` folder using any standard web server (like Nginx, Apache, or IIS) or configure the Go backend to locally serve these static files alongside the API endpoints.

---

8. Default Admin Login

Upon the very first launch, if you register a new account, it might default to a regular user. You can manually escalate the first user to an `admin` directly via MySQL:

```sql
UPDATE users SET role = 'admin' WHERE id = 1;
```

Once logged in as the admin, you can manage, promote, and approve all future staff accounts directly from the front-end **Staff & Roles** dashboard.